W

WREXLYN

The Local-First AI Orchestration Workspace

Features, differentiation, expansion potential, and category comparison — a complete product briefing.

23	**7**	**13**	**6**
built-in tools	model provider paths	orchestration layers	reliability check-states

01

THE PROBLEM

A chat window with tool access is not a controlled workspace

Giving a model file and shell access answers a different question than making its output trustworthy. Most agentic tools optimize generation speed and leave verification, recovery, and memory as an afterthought — or skip them entirely.

Confident, not correct

A model narrates its work fluently regardless of whether the change actually works — tone is not evidence.

Retry without recovery

A failed attempt is usually just retried verbatim, with no independent diagnosis of why it failed.

Documents are pretend-checked

A generated Word/PPT/Excel file is rarely inspected structurally — “didn’t throw” is treated as “correct.”

Model lock-in or cloud lock-in

IDE-embedded tools tie you to one editor and model; hosted agents move your code to infrastructure you can’t inspect.

No memory across sessions

Preferences, working commands, and lessons learned are re-derived from scratch on every new conversation.

No audit trail

When something breaks, there is often no record of what changed, what was checked, or how to undo it.

02

THE THESIS

Models are replaceable infrastructure. The workspace is the product.

Wrexlyn is a single local Node.js process — not a browser extension, not a hosted SaaS — that gives any LLM provider a governed loop for context, tools, permissions, verification, and memory. Everything it touches lives on your machine.

LOCAL-FIRST

code, sessions, keys and generated files never leave the machine unless the chosen model provider needs the message text

DETERMINISTIC CONTROL LOOP

risk-scored, snapshotted, checked and scored — so honesty doesn’t depend entirely on model care

MODEL-AGNOSTIC

6 integrated cloud providers, any custom endpoint, or a fully local model server — same harness either way

Core product thesis. Durable value comes from the workspace that gives models context, tools, reusable skills, permission boundaries, verification and project knowledge that compounds — not from any single model.

03

THE OPERATING CONTRACT

Six commitments the model provider never changes

The underlying model may change by task, price, privacy requirement or customer policy. What Wrexlyn commits to around it does not.

01

Understand before acting

Repo structure, ranked lexical search, extracted symbols and Git history narrow the working set before any edit.

02

Control every mutation

Typed tools, path validation, risk classification and pre-change snapshots gate every effect on the machine.

03

Prove completion

Configured builds/tests/lint drive a six-state outcome; structural gates check non-code deliverables the same way.

04

Recover deliberately

Independent critique, bounded repair, recurrence detection and NCP replace infinite, uninformed retries.

05

Preserve customer choice

Integrated providers, any compatible URL, and local runtimes like Ollama share one execution harness.

06

Deliver an audit trail

Tool activity, evidence, outcome state, confidence, transactions and rollback data explain what happened.

04

SYSTEM DESIGN

One process, five layers, zero required external services

The browser UI talks to a single Node.js process over one WebSocket. Every tool call, permission prompt and streamed token flows through the same typed protocol — no bundler, no build step for the frontend, no mandatory backend service.

LAYER	WHAT IT IS
Agent loop	Drives the model, dispatches tool calls, runs the reliability pipeline, persists sessions per project.
Tools	23 typed tools — file I/O, search, shell, Word/PPT/Excel/PDF generation & reading, web fetch — each risk-classified.
Provider layer	One shared OpenAI-compatible HTTP/SSE client backs every integrated provider and the Custom / Local Model option.
Web server	Static file serving with no build step (UI edits are live on refresh), the WebSocket protocol, and local-only REST endpoints.
Persistence	Per-project sessions, transaction/audit log, learned preferences and saved skills under .coding-agent/; machine-wide settings under the user’s home directory.

05

THE ORCHESTRATION STACK — PART I

Thirteen implemented capability layers, understand → execute

01–02 Secure execution foundation

Local-first runtime, authenticated web access, strict path validation, protected secret storage, SSRF-resistant fetching, scrubbed MCP environments.

03 Six-state verification engine

verified / verified-with-warnings / unverified / failed / blocked / not-applicable — required and advisory checks stay distinct.

04 Checkpoints and rollback

Binary-safe per-file snapshots and Git-plumbing workspace checkpoints. Staleness checks stop a rollback overwriting newer work.

05 Project intelligence

Lexical ranking, symbol extraction, repo structure and Git-history signals find the code that matters before editing.

06 Evidence and consistency

Claims cite captured evidence; a consistency layer flags contradictions across generated artifacts.

07 Professional artifact engine

One structured document model compiles to Markdown/HTML/PDF and drives native Word/PPT/Excel generation.

05

THE ORCHESTRATION STACK — PART II

...execute → verify → adapt for next time

08 Versioned skills platform

Reusable capabilities packaged with manifests, instructions, scripts, tests and evals — governed product IP, not loose prompts.

09 Enterprise-grade MCP connectors

Local stdio and remote Streamable HTTP servers, OAuth 2.1 flows, per-server policy, anti-injection framing for external content.

10 Best-of-N isolated agents

Multiple agents attempt the same hard task in real Git worktrees; candidates are independently verified and ranked.

11 Product execution console

Sessions, project selection, plans, live tool activity, permissions, evidence and parallel-run results in one workspace.

12 Local enterprise governance

Control applied at the customer-controlled runtime, not a mandatory hosted multi-tenant plane — custody stays with the customer.

13 Reproducible product evaluation

Task success, verification integrity, regressions, recovery uplift, safety and cost — pinned to build, model and environment.

06

THE RELIABILITY RUNTIME

The actual differentiator: a control loop wrapped around the model

Every mutating action — writing a file, running a command — passes through the same six-step pipeline. The whole turn is checked, scored, and can be reverted.

Risk gate

Low/med/high scored before running. Destructive patterns can never be pre-approved.

Snapshot

Every touched file captured beforehand — one button restores exact prior content.

Verification

Real configured build/test/lint checks. A failure is evidence, not a vibe.

Independent critic

A second model call, no tools, no stake in the outcome, reviews the result.

Bounded repair

Up to 3 automatic rounds, with stuck-loop detection on identical output.

Confidence score

An evidence-based 0–100 score and outcome end every turn.

Honest about scope. This reduces accidental damage from a model doing something it shouldn’t. It is not a sandbox — a genuinely compromised model with shell access is a different, larger problem a permission prompt alone doesn’t solve.

07

VERIFICATION ENGINE

Six honest outcomes, not one confident tone

Required checks (a real build/test) and advisory checks (an independent critic’s opinion) are kept distinct, so a confident model response can never silently overwrite objective test evidence.

STATE	WHAT IT MEANS
Verified	Every required check ran and passed — objective evidence, not a self-report.
Verified, with warnings	Required checks passed; an advisory signal (e.g. the critic) flagged something worth a look.
Unverified	Changes were made but nothing applicable ran to check them.
Failed	A required check ran and did not pass.
Blocked	The turn stopped before completion — a permission denial or a hard error.
Not applicable	No mutating action occurred; there was nothing to verify.

08

CHECKPOINTS AND ROLLBACK

Every change is reversible by construction, not by hope

Binary-safe per-file snapshots

Captured before any write, edit, or delete — non-UTF-8 content restores byte-exact.

Git-plumbing workspace checkpoints

Creates, edits, deletes, renames, and file-mode changes are all tracked, not just text diffs.

Staleness checks

A rollback refuses to silently overwrite a file the user modified after the transaction finished.

One-click revert

A failed or unwanted turn restores every changed file to its exact prior state — no manual undo.

09

PROJECT INTELLIGENCE

Grounded in the actual repository, not a guessed architecture

Search is a first-class subsystem, not a side effect of the model's training data — the agent inspects relevant code instead of inferring structure from filenames or a partial prompt.

Ranked lexical search

Term-frequency relevance across the project, not embedding-based semantic search — stated plainly, not oversold.

Symbol extraction

Functions, types and classes are indexed for precise lookups, answered directly rather than by scanning raw text.

Repository structure

The top-level file listing, package.json, and README load automatically on startup — no manual directory listing needed just to orient.

Git-history signals

Recent commits and change patterns inform what's actually active in a codebase, not just what exists.

10

EVIDENCE AND CONSISTENCY

Claims that cite sources, and contradictions that get flagged

Figures and facts captured during a session can be cross-checked against later claims — useful for diligence packs, technical reports and any multi-document deliverable, not just code.

Evidence capture

Captures a labeled figure or fact during the session, with its source, for later reference.

Conflict detection

A new claim that numerically contradicts a previously recorded one is flagged, not silently overwritten.

Cross-artifact consistency

The same fact repeated across several generated documents is checked for agreement, not just internal correctness.

11

DOCUMENT GENERATION

Real, structured files — not a generic file write pretending to be a document generator

Word, PowerPoint and Excel output as genuine, structured files, with formatting as a first-class input rather than an afterthought.

Word

Inline bold/italic/underline/strike, numbered & nested lists, real images sized from their own aspect ratio, a genuine auto-updating table of contents, per-cell table formatting, custom accent color.

PowerPoint

Native, PowerPoint-editable charts, high-end tables, and two infographic-native layouts. Deliberately no decorative accent-stripe under titles — the single most recognizable AI-deck tell.

Excel

Live, recalculating formulas rather than baked-in numbers, per-column currency/percent/date formats, merged cells, autofilter.

Every one of these is verified by unzipping the actual generated file and asserting on its real OOXML — bold runs present, numbering XML present, image relationships registered, a real chart part present — not just "the call didn't throw."

12

POWERPOINT, DEEP DIVE

Beyond title-and-bullets: four layouts built for what the content actually is

Native charts, not screenshots

Bar, line, pie and doughnut charts render as real PowerPoint chart objects — clickable and re-themeable, with themed axes, gridlines and labels.

High-end tables

Per-column width control instead of an automatic even split, plus a highlighted-row option for the one line that matters.

Stats layout

A handful of headline numbers as large bordered callout boxes — its own standalone layout for a slide whose entire job is 3–4 big metrics.

Timeline layout

Numbered steps connected by a line, deliberately without a card background, so a roadmap reads as a connected flow.

0

decorative accent stripes — by design

2ND PASS

generator revision, specifically to stop defaulting to plain bullets

13

DOCUMENT QUALITY GATE

A generated document is a claim — this checks it before it’s trusted

Code is verified by running it. A .docx/.pptx/.xlsx can’t be “run,” so a deterministic checker inspects the file itself — structurally, not by asking a second model whether it looks right.

Structural checks

Placeholder text, mismatched table rows, headers with zero data — plain code, not a model opinion.

Fails closed

Runs immediately before the file commits to disk; the model retries in the same turn.

Soft warnings

A 6+ bullet slide or an unformatted spreadsheet column reports alongside success, not blocking it.

Real confidence

A clean pass is scored genuinely verified — not the old flat 60% “unverified changes.”

What this can’t fix. It catches defined structural defects regardless of model. It cannot judge factual accuracy or writing quality — review every deliverable before relying on it.

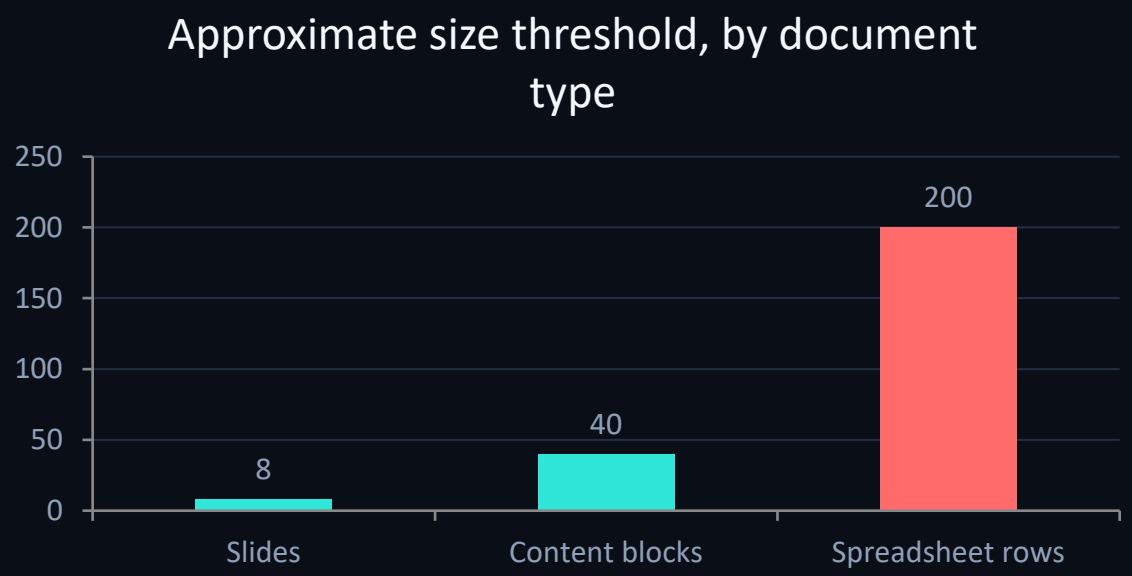
14

SCRIPT-BASED GENERATION

For documents too large for a single completion

Every provider caps one completion around 8,000 tokens. Past a size threshold, Wrexlyn has the model write a compact Node.js script instead of one giant JSON call — loops express repetitive structure far more compactly than expanded JSON ever could.

Script-based generation kicks in past:



Real module scope

Runs with NODE_PATH pointed at Wrexlyn's own modules — no source concatenation, correct stack-trace line numbers.

Same isolation as shell exec

Routes through the identical forked-process isolation; always high-risk, never a standing "always allow."

A curated kit, not a blank slate

Small helper packages port the theme, icon badges and chart presets so a script still looks Wrexlyn-made.

Same gate, post-render

The rendered file is re-opened and checked exactly like the JSON path — same retry loop, same turn.

15

MODEL PROVIDERS

Interchangeable infrastructure, from zero-key to fully local

Use an integrated provider, connect any compatible hosted endpoint by URL, or run the model entirely on the same machine — Wrexlyn's planning, tools, permissions and verification remain the control layer above all of them.

PROVIDER	KEY REQUIRED	NOTES
Kilo	No key	Default. Free, anonymous gateway auto-routing to an available upstream model — capped at 200 req/hour/IP.
Groq	Free key	Fast inference, generous free tier.
OpenRouter	Free key	Access to the rotating ":free"-tier community model lineup.
Google Gemini	Free key	No credit card, via Google AI Studio.
Cerebras	Free key	1M tokens/day free tier.
Mistral	Free key	Free tier, phone verification required.
Custom / Local	Optional	Any OpenAI-compatible endpoint — a paid API, or a local server like Ollama, LM Studio, llama.cpp, vLLM.

16

ADAPTIVE PROJECT MEMORY

Learns in a bounded, inspectable way — not by retraining weights

Wrexlyn persists useful project facts, stated preferences, reusable workflows and recurring quality failures. It does not rewrite its own source code or fine-tune a model.

Remembers stated preferences

A dedicated memory tool persists a standing instruction — formatting, tone, workflow — project-scoped or global, the moment it's stated.

Saves reusable skills

A dedicated skill-saving tool persists a genuinely reusable multi-step pattern to disk; a matching recall tool fetches its full steps on demand, keeping the prompt small.

Applies immediately

Learning a preference or lesson rebuilds the system prompt in the same turn — not only on the next session.

Claim boundary. *This is adaptive project memory, not autonomous model training. Every learned item changes future context, not the underlying model.*

17

VERSIONED SKILLS PLATFORM

Repeatable expertise as governed product IP, not loose prompts

Reusable capabilities are packaged with manifests, instructions, scripts, tests and evals — permissions are declared and previewed before any skill script can execute.

check

Versioned

Manifests and packages preserve an inspectable history of what changed and when.

lock

Permission-aware

Script execution stays subject to the exact same explicit tool permissions as everything else.

check

Testable

Packages can include sample inputs and deterministic checks, not just instructions.

Review before use. A skill can contain executable instructions. Inspect its manifest and script preview, verify its source, and apply least privilege before running it.

18

SELF-IMPROVING SKILL LOOP

Skills that track their own record, and improve only with your sign-off

Every skill recall ties back to that turn’s real outcome and confidence score. A background judge — a pure heuristic filter, then a model call — proposes a new skill or a revision from it.

Tracks real outcomes

Usage, success rate and recent-use history accumulate on every skill — the same signal that already computes the turn’s confidence score.

Proposes, never applies

A new skill or a revision is a pending proposal in the Skills tab — nothing changes what the agent does until it’s explicitly accepted.

Compounds across projects

Accepting the same skill in 2+ projects proposes promoting it to an account-wide library, ready for a brand-new project’s first turn.

Claim boundary. *This still only produces a proposed skill file for review — never a change to Wrexlyn’s own prompts or source, never applied without your accept.*

19

MCP CONNECTORS

Enterprise-grade protocol support, with governance, not a bare pipe

Local stdio and remote Streamable HTTP MCP servers, including OAuth 2.1 flows, per-server permission policy, and anti-injection framing — external content is treated as data, never silently promoted to trusted instructions.

A CURATED CONNECTOR GALLERY, OR ADD ANY SERVER MANUALLY:

Gmail

Search, read, and send email in your Gmail account.

Google Drive

Search, read, and manage files in Google Drive.

Google Calendar

Read and manage calendar events.

OneDrive / SharePoint

Files across OneDrive and SharePoint (tenant-scoped).

Dropbox

Search, read, and manage files in Dropbox.

Any MCP server

Filesystem, git, memory, or a custom server — add by command + args, no JSON editing required.

20

BEST-OF-N ISOLATED AGENTS

Attempt the hard task multiple ways, in real isolation

Multiple agents attempt the same difficult task in real Git worktrees. Candidates are independently verified and ranked; the selected result applies cleanly, without mixing intermediate edits into the working tree.

Fork N worktrees

Each attempt gets a genuinely isolated Git worktree, not a shared directory.

Run in parallel

Every attempt runs the full agent loop — tools, permissions, verification — independently.

Independently verify

Each candidate is scored on its own evidence, not compared by vibes.

Rank and apply

The best-verified candidate merges cleanly; the rest are discarded without touching your tree.

REAL WORKTREES

genuine git-worktree isolation, not a simulated sandbox

SHARED PACING GATE

concurrent attempts don’t all burst the keyless provider’s rate cap at once

21

THE NISHANT CONVERGENCE PROTOCOL

Closing the self-correction gap for weak and free models

A weak model is rarely bad at a first attempt — it's bad at self-correcting once that attempt fails. NCP formalizes the fix as a named, versioned, unit-tested algorithm, invoked only once a plain repair has already failed once.

Divergent diagnosis

Two independently-framed root-cause diagnoses generated in parallel from the same evidence.

Pairwise adjudication

A dedicated comparative call judges the two against each other — more reliable than scoring either alone.

Lesson-steered regen

Ranked learned lessons foreground as explicit constraints inside both diagnosis prompts.

Convergence score

A strict refinement of the outcome score — unchanged on the common, zero-repair path.

```
A bounded scoring adjustment weighs repair rounds spent, whether the ensemble reached a clear adjudicated winner,
and whether a previously-learned failure recurred anyway.
```

Said plainly. NCP does not make a small model reason like a large one. It closes the reliability gap on tasks with an external check — not a claim of raw intelligence uplift.

22

PRODUCT EXECUTION CONSOLE

Desktop, terminal, and phone — the same session, everywhere

Desktop-grade web UI

Sessions, project selection, plans, live tool activity, permission decisions, evidence and verification in one workspace. Windows installer (Inno Setup) and a Linux launch script; auto-update checks on every launch.

Terminal REPL

Same agent, same tools, same y/a/n permission prompts, for anyone who'd rather not leave the shell.

Mobile, same LAN

A QR code opens the running session on a phone's browser, installable to the home screen as a lightweight PWA. Every connection — local or LAN — still requires the per-run auth token.

ONE CODEBASE

identical agent loop and tool surface across web, CLI and mobile — no feature gap between them

127.0.0.1 BY DEFAULT

the web UI is not reachable from the network unless —lan is explicitly passed

23

SECURITY AND PRIVACY

Data residency you can actually verify

Data residency

Everything — code, sessions, generated files, API keys — stays on the machine. Nothing goes anywhere except the model provider you chose, for the messages you sent it.

Sandbox boundary

File tools are confined to the chosen project root; a path resolving outside it is rejected before touching disk.

Key storage

Provider credentials go through the platform-aware secret store (DPAPI / Keychain / Secret Service), with a disclosed plaintext fallback only when none is available.

Shell execution

Delegated to a separate service and gated by the risk/permission system — not a container or VM boundary, stated plainly.

Undo

Best-effort checkpoints before every supported mutating action — not a replacement for version control or independent backups.

24

EVALUATION AND PROOF STANDARD

Measuring the system, not showcasing the model

A serious coding agent must be evaluated as a complete system. A reproducible run pins the Wrexlyn build, model, endpoint, task set and verification command — and results are reported by model tier, so a frontier model can't hide a weak harness.

DIMENSION	PRIMARY METRIC	DECISION IT SUPPORTS
Task completion	Resolved rate, pass@1	Does the complete system produce correct patches?
Verification integrity	Precision of the verified state	Can a reviewer trust the completion label?
Regression control	Previously-passing checks broken	Does verification prevent locally-correct, globally-harmful changes?
Recovery uplift	Success after first failure	Does orchestration add value beyond the base model?
Operational safety	Unsafe actions proposed / blocked / executed	Can autonomy operate inside an acceptable risk envelope?
Economics	Cost and time per verified result	Which deployment produces the best accepted-work economics?

Current evidence boundary. Wrexlyn does not claim an official third-party benchmark score today — automated tests cover internal mechanisms; external validation is sequenced deliberately, not skipped.

25

COMPETITIVE POSITIONING

Four categories answer the same question differently

This is a map of the actual trade-off, not a claim that other categories are bad at what they optimize for.

CATEGORY	OPTIMIZES FOR	WHAT YOU GIVE UP
Inline autocomplete assistants	Speed of a single suggestion, in-editor	No multi-file planning, no verification loop, no memory across sessions
IDE-embedded agentic assistants	Convenience — one tool, one subscription	Usually a fixed model, usually a subscription regardless of upstream free tiers
Hosted autonomous SaaS agents	Fire-and-forget delegation	Code and prompts leave your machine; verification, if any, runs on infrastructure you can’t inspect
Other local/terminal agents	Also local, also often model-agnostic	Typically no independent critic, no formalized confidence score, no document quality gate

Wrexlyn’s bet: verified reliability compounds where raw generation speed does not — especially once free or lower-cost models, not always the frontier ones, are doing the generating.

26

PEER COMPARISON, BY CAPABILITY

Reflects documented category patterns, not named-vendor scoring

CAPABILITY	INLINE AUTOCOMPLETE	IDE-EMBEDDED AGENT	HOSTED SAAS AGENT	OTHER LOCAL AGENTS	WREXLYN
Full multi-file agent loop	—	✓	✓	✓	✓
Editor- and model-agnostic	—	—	—	~	✓
Runs entirely on your machine	~	~	—	✓	✓
Independent critic step	—	—	~	—	✓
Structural document quality gate	—	—	—	—	✓
Formalized numeric confidence score	—	—	—	—	✓
Per-file checkpoint + one-click rollback	~	~	—	~	✓
Local model server support	—	—	—	~	✓
Best-of-N isolated parallel attempts	—	—	~	—	✓
Versioned, permission-gated skills platform	—	~	~	—	✓

27

WHY THE COMPARISON HOLDS UP

Evidence, named mechanisms, and disclosed limits

check

Evidence, not claims

Retry/backoff behavior, verification outcomes, checkpoints, evidence handling and document quality controls are all backed by automated tests in the codebase — a different kind of claim than marketing copy alone.

star

Named, versioned mechanisms

“The reliability runtime,” “the document quality gate,” “the Nishant Convergence Protocol” are specific, citable names for specific code — not a marketing umbrella over whatever the model happened to do that day.

warning

Disclosed limits, not just wins

Every differentiation claim in this deck has a matching honest-scope callout for what it does not do. A tool that discloses its own ceiling is more trustworthy at the claims it does make.

28

WHERE THIS GOES NEXT

Roadmap items and adjacent markets, tracked rather than hidden

NEAR-TERM ROADMAP

- Container-level sandboxed execution for the use cases where host-level trust isn’t acceptable
- Force-kill for a running shell command mid-execution, not just a stop on the loop continuing
- Prose-quality scoring — the current gate is deliberately structural, not writing-quality-aware yet
- Team-shared skill library — single-user, cross-project promotion now ships; multi-user distribution is next
- External validation sequence: private pilot → same-model ablations → SWE-bench Verified, Terminal-Bench 2, BFCL, SWE-Lancer

ADJACENT EXPANSION SURFACE

Diligence & analysis packs

Evidence + consistency checking already fits diligence, technical reports, and multi-document deliverables beyond code.

Connector ecosystem

The MCP gallery (Gmail, Drive, Calendar, OneDrive, Dropbox) turns a coding agent into a broader orchestration workspace.

Regulated / local-governance buyers

Local-first custody is the pitch for customers who cannot send code or prompts to third-party infrastructure.

Skills as a distribution surface

A versioned, testable skills platform is the seed of a governed capability marketplace, not just a personal shortcut.

The model is replaceable. The workspace compounds.

- One local process. Any model. Governed tools, verified results, memory that compounds.
- 23 built-in tools, 7 provider paths, 13 orchestration layers, 6 verification states — all inspectable, all local.
- Evidence-based differentiation, disclosed limits, and a roadmap tracked in the open.

Let’s talk.

A live walkthrough, source review, and full technical deep-dive are available under NDA.